

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

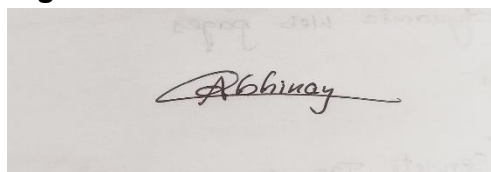
Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- Analyze the CSS properties responsible for layout overlapping issues.
- Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I G.Abhinay(192373062) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation
---------------	----	---	--	--	----------------------------------

Experiment 1: Responsive Layout Design

Aim:

To design and implement a responsive webpage using CSS Flexbox and Media Queries that adapts to mobile, tablet, and desktop screen sizes while maintaining proper alignment and spacing.

(a) Analysis of the Existing Layout Problems

A retail website may fail to be responsive due to the following reasons:

1. **Fixed Width Layout**
 - Uses pixel values (e.g., width:1200px) instead of percentages or flexible units.
2. **No Media Queries**
 - The layout remains the same on all screen sizes.
3. **Images Not Responsive**
 - Images exceed screen width and break the layout.
4. **Improper Navigation**
 - Menu items overlap on smaller screens.
5. **No Flexible Containers**
 - Uses float positioning instead of Flexbox or Grid.
6. **Poor Spacing**
 - Margins and padding are fixed, causing uneven spacing.
7. **Viewport Not Defined**
 - Missing viewport meta tag prevents proper scaling on mobile devices.

b) Responsive Webpage Design

HTML (index.html)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Retail Website</title>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>My Retail Store</h1>
```

```
</header>
```

```
<nav>
```

```
<a href="#">Home</a>
```

```
<a href="#">Products</a>
```

```
<a href="#">Offers</a>
```

```
<a href="#">Contact</a>
```

```
</nav>
```

```
<div class="container">
```

```
<div class="sidebar">
```

```
<h2>Categories</h2>
```

```
<p>Electronics</p>
```

```
<p>Fashion</p>
```

```
<p>Furniture</p>
```

```
</div>
```

```
<div class="content">
```

```
<h2>Welcome</h2>
```

```
<p>
```

Our online retail website offers quality products at affordable prices.

The responsive layout automatically adjusts according to different devices.

```
</p>
```

```

```

```
</div>
```

```
</div>
```

```
<footer>
© 2026 My Retail Store
</footer>
```

```
</body>
</html>
```

CSS (style.css)

```
*{
margin:0;
padding:0;
box-sizing: border- box;
font - family: Arial;
}
```

```
body {
background: #f4f4f4;
}
```

```
header {
background: #0077cc;
color: white;
padding:20px;
text- align: center;
}
```

```
nav {
display: flex;
justify- content: center;
background: #222;
}
```

```
nav a{
color:white;
padding:15px;
text-decoration:none;
}
```

```
nav a:hover{  
background:#555;  
}
```

```
.container{  
display:flex;  
padding:20px;  
gap:20px;  
}
```

```
.sidebar{  
flex:1;  
background:white;  
padding:20px;  
}
```

```
.content{  
flex:3;  
background:white;  
padding:20px;  
}
```

```
.content img{  
width:100%;  
height:auto;  
margin-top:15px;  
}
```

```
footer{  
background:#0077cc;  
color:white;  
text-align:center;  
padding:20px;  
margin-top:20px;  
}
```

(c) Media Queries for Different Screen Sizes

```
/* Tablet */
```

```
@media(max-width:992px){

.container{
flex-direction:column;
}

nav{
justify-content:space-around;
}

}

/* Mobile */

@media(max-width:600px){

nav{
flex-direction:column;
align-items:center;
}

nav a{
width:100%;
text-align:center;
border-bottom:1px solid white;
}

header{
font-size:18px;
}

.content,
.sidebar{
padding:15px;
}

}
```

(d) Spacing, Alignment, and Distribution Techniques

The following CSS properties improve the webpage appearance:

```
.container{  
display:flex;  
gap:20px;  
justify-content:space-between;  
align-items:flex-start;  
}
```

```
nav{  
display:flex;  
justify-content:center;  
align-items:center;  
}
```

```
.content{  
padding:20px;  
line-height:1.8;  
}
```

```
.sidebar{  
padding:20px;  
}
```

```
img{  
max-width:100%;  
height:auto;  
}
```

Techniques Used

- Flexbox for flexible layout
- gap for equal spacing
- padding for internal spacing
- justify-content for horizontal alignment
- align-items for vertical alignment
- max-width:100% for responsive images

(e) Evaluation

Results

- The webpage automatically adjusts for desktop, tablet, and mobile devices.
- Navigation becomes vertical on smaller screens.
- Images resize without overflowing.
- Sidebar stacks below the navigation on smaller devices.
- Proper spacing improves readability and visual consistency.

Advantages

- Faster loading on mobile devices
- Better user experience
- Easy to maintain
- Compatible with modern browsers
- Flexible content arrangement

Suggested Improvements

1. Add a hamburger menu for mobile navigation.
2. Use CSS Grid for more complex page layouts.
3. Optimize images using modern formats like WebP.
4. Improve accessibility by adding ARIA labels and keyboard navigation.
5. Add animations and hover effects for better user interaction.

Output

- **Desktop:** Header → Horizontal Navigation → Sidebar + Content → Footer
- **Tablet:** Header → Navigation → Sidebar (stacked) → Content → Footer
- **Mobile:** Header → Vertical Navigation → Content → Footer

The responsive design ensures proper alignment, spacing, and usability across desktop, tablet, and mobile devices.

Experiment 2: Form Validation using JavaScript

Aim

To design and implement a registration form with JavaScript validation using Regular Expressions (Regex) for email and phone number fields, ensuring only valid data is submitted.

(a) Analysis of Existing Form Validation Issues

The registration form may accept invalid data due to the following reasons:

1. **No Client-Side Validation**
 - The form submits even when fields contain incorrect values.
2. **Improper Email Validation**
 - Invalid email formats like abc@, user.com, or @gmail.com are accepted.
3. **Invalid Phone Number**
 - Letters, symbols, or phone numbers with incorrect length are accepted.
4. **No Real-Time Feedback**
 - Users only know about errors after submitting the form.
5. **Poor User Experience**

- Users must repeatedly submit the form to identify mistakes.
6. **Lack of Security**
- Invalid or malicious input may reach the server if server-side validation is absent.

(b) HTML Design for Input Fields

index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Registration Form Validation</title>

<style>

body{
font-family:Arial;
background:#f4f4f4;
}

.container{
width:350px;
margin:50px auto;
background:white;
padding:20px;
border-radius:8px;
box-shadow:0px 0px 10px gray;
}

input{
width:100%;
padding:10px;
margin:10px 0;
}

button{
width:100%;
```

```
padding:10px;
background:#007BFF;
color:white;
border:none;
cursor:pointer;
}
```

```
.error{
color:red;
font-size:14px;
}
```

```
.success{
color:green;
font-size:14px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h2 align="center">Registration Form</h2>
```

```
<form id="myForm">
```

```
<label>Email</label>
```

```
<input
type="email"
id="email"
placeholder="Enter Email"
required>
```

```
<div id="emailError" class="error"></div>
```

```
<label>Phone Number</label>

<input
type="tel"
id="phone"
placeholder="Enter Phone Number"
maxlength="10"
required>

<div id="phoneError" class="error"></div>

<button type="submit">
Register
</button>

</form>

</div>

<script src="script.js"></script>

</body>
</html>
```

(c) JavaScript Validation Using Regular Expressions

script.js

```
const email=document.getElementById("email");
const phone=document.getElementById("phone");

const emailError=document.getElementById("emailError");
const phoneError=document.getElementById("phoneError");

const form=document.getElementById("myForm");

const emailPattern=/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;

const phonePattern=/^[6-9]\d{9}$/;
```

```
// Email Validation
email.addEventListener("keyup",function(){

if(emailPattern.test(email.value))
{
emailError.innerHTML="Valid Email";
emailError.className="success";
}
else
{
emailError.innerHTML="Invalid Email Address";
emailError.className="error";
}
});

// Phone Validation
phone.addEventListener("keyup",function(){
if(phonePattern.test(phone.value))
{
phoneError.innerHTML="Valid Phone Number";
phoneError.className="success";
}
else
{
phoneError.innerHTML="Enter Valid 10-digit Phone Number";
phoneError.className="error";
}
});

// Form Submission
form.addEventListener("submit",function(e){
if(
!emailPattern.test(email.value)
||
!phonePattern.test(phone.value)
)
{
alert("Please Enter Valid Details");
e.preventDefault();
}
});
```

(d) Real-Time Error Message Mechanism

Email Validation

- Displays "Valid Email" in green when the email format is correct.
- Displays "Invalid Email Address" in red when the format is incorrect.

Phone Number Validation

- Displays "Valid Phone Number" in green for a valid 10-digit number.
- Displays "Enter Valid 10-digit Phone Number" in red for invalid input.

Form Submission

- Prevents form submission until both fields are valid.
- Displays an alert prompting the user to correct invalid inputs.

(e) Evaluation

Result

The validation system successfully:

- Validates email format using Regular Expressions.
- Validates 10-digit phone numbers starting with digits **6–9**.
- Displays real-time validation messages.
- Prevents invalid form submission.
- Improves data accuracy and user experience.

Advantages

- Reduces invalid data submission.
- Provides immediate feedback to users.
- Enhances usability with real-time validation.
- Uses simple and efficient Regular Expressions.
- Easy to maintain and extend.

Suggested Enhancements

1. Implement **server-side validation** in addition to client-side validation.
2. Sanitize user input to prevent SQL Injection and Cross-Site Scripting (XSS).
3. Add password strength validation.
4. Include OTP verification for phone numbers.
5. Provide accessible error messages using ARIA attributes.
6. Disable the submit button until all inputs are valid.
7. Use input masking for phone number formatting.

Output

Valid Email: student@gmail.com → Valid Email

Invalid Email: student@ → Invalid Email Address

Valid Phone: 9876543210 → Valid Phone Number

Invalid Phone: 12345abcd → Enter Valid 10-digit Phone Number

Form Submission: Only allowed when both email and phone number are valid.

Experiment 3: CSS Layout Debugging

Aim

To identify and fix CSS layout overlapping issues by using the CSS Box Model, proper positioning techniques, and responsive layout methods such as Flexbox and Grid.

(a) Analysis of CSS Properties Responsible for Layout Overlapping

The webpage layout may overlap due to the following reasons:

1. Fixed Width and Height
 - Using fixed values (e.g., width:1200px;) prevents the layout from adapting to different screen sizes.
2. Absolute Positioning
 - Elements positioned with position: absolute; may overlap other content when the screen size changes.

3. Floating Elements

- Using float:left or float:right without clearing floats can cause layout issues.

4. Improper Z-index

- Incorrect z-index values may cause elements to appear on top of each other.

5. Missing Responsive Design

- Lack of media queries prevents the layout from adjusting on smaller screens.

6. Overflow Issues

- Content may exceed its container when overflow is not handled properly.

(b) CSS Box Model

The CSS Box Model consists of four parts:

- i) Margin
- ii) Border
- iii) Padding
- iv) Content

Components

Component Description

Content	The actual text, images, or other content inside the element.
Padding	Space between the content and the border.
Border	The outline surrounding the padding and content.
Margin	Space outside the border that separates elements.

Importance

- Prevents elements from touching each other.
- Controls spacing and alignment.
- Improves readability.

- Maintains a consistent layout.

(c) Debugging Using Positioning Techniques

HTML (index.html)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>CSS Layout Debugging</title>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<header>Header</header>
```

```
<nav>
```

Home | About | Services | Contact

```
</nav>
```

```
<div class="container">
```

```
<div class="sidebar">
```

Sidebar

```
</div>
```

```
<div class="content">
```

Main Content Area

```
</div>
```

```
</div>
```

```
<footer>
```

Footer

```
</footer>
```

```
</body>
```

```
</html>
```

CSS (style.css)

```
*{  
margin:0;  
padding:0;  
box-sizing:border-box;  
}  
body{  
font-family:Arial;  
background:#f4f4f4;  
}  
header,nav,footer{  
background:#0077cc;  
color:white;  
text-align:center;  
padding:20px;  
}  
.container{  
display:flex;  
gap:20px;  
padding:20px;  
}  
.sidebar{  
flex:1;  
background:#dddddd;  
padding:20px;  
}  
.content{  
flex:3;  
background:white;  
padding:20px;  
}  
footer{
```

```
margin-top:20px;  
}
```

(d) CSS Modification to Prevent Overlapping

Responsive CSS

```
/* Tablet */  
@media(max-width:768px){  
  .container{  
    flex-direction:column;  
  }  
}  
/* Mobile */  
@media(max-width:480px){  
  header,  
  nav,  
  footer{  
    font-size:18px;  
  }  
  .container{  
    padding:10px;  
  }  
  .sidebar,  
  .content{  
    width:100%;  
  }  
}
```

Positioning Techniques Used

Technique	Purpose
position: relative;	Positions an element relative to its normal position.
position: absolute;	Positions an element relative to its nearest positioned ancestor (used carefully to avoid overlap).
display: flex;	Creates flexible layouts that adjust automatically.
display: grid;	Creates structured two-dimensional layouts.
box-sizing: border-box;	Includes padding and border in the element's total width and height.

(e) Evaluation

Result

The corrected layout:

- Prevents overlapping during browser resizing.
- Aligns content properly using Flexbox.
- Adjusts automatically for desktop, tablet, and mobile screens.
- Maintains consistent spacing using the CSS Box Model.
- Improves readability and usability.

Advantages

- Responsive on all screen sizes.
- Easy to maintain and modify.
- Better user experience.
- Proper spacing and alignment.
- Modern layout using Flexbox.

Best Practices

1. Use **Flexbox** or **CSS Grid** instead of floats for layouts.
2. Avoid excessive use of position: absolute.
3. Use relative units such as %, em, rem, and vw instead of fixed pixel widths.
4. Apply box-sizing: border-box; to simplify sizing calculations.

5. Add media queries for different screen sizes.
6. Test layouts on mobile, tablet, and desktop devices.
7. Optimize images with max-width: 100%; and height: auto;.
8. Maintain consistent margins and padding throughout the webpage.

Output

- **Desktop:** Header → Navigation → Sidebar and Content displayed side by side → Footer.
- **Tablet:** Sidebar and Content stack vertically without overlapping.
- **Mobile:** All sections are displayed in a single column with proper spacing and alignment.

The corrected CSS layout ensures that the webpage remains responsive, visually consistent, and free from overlapping across different screen sizes.